JPA e **Hibernate** são tecnologias do ecossistema Java usadas para trabalhar com persistência de dados em bancos de dados relacionais por meio do conceito de **ORM** (*Object-Relational Mapping* ou Mapeamento Objeto-Relacional), que transforma tabelas do banco em objetos na linguagem Java. [1, 2, 3]

Apesar de frequentemente serem citados juntos, eles têm papéis diferentes:

1. O que é o JPA?

- **O que é:** O **JPA** (*Java Persistence API* — hoje também conhecido como *Jakarta Persistence*) **não é um código pronto**, mas sim uma **especificação** (um conjunto de regras e interfaces) criada pela Oracle/Java EE. [1, 2, 3]
- **Para que serve:** Ele dita *como* o mapeamento entre objetos Java e tabelas do banco deve ser feito (por exemplo, definindo que uma classe com a anotação `@Entity` representa uma tabela). [1, 2, 3]
- **Importante:** Você **não consegue rodar** apenas o JPA, pois ele é só o "manual de instruções" (a interface). [1, 2]

2. O que é o Hibernate?

- **O que é:** O **Hibernate** é um **framework** de ORM. [1, 2]
- **Para que serve:** Ele é uma **implementação** prática da especificação JPA. Ou seja, ele lê as regras do JPA e faz o trabalho pesado de verdade: gera os comandos SQL automaticamente, gerencia transações, abre conexões e converte os dados do banco em objetos Java e vice-versa. [1, 2, 3, 4]
- **Outras implementações:** Embora o Hibernate seja o mais famoso e usado do mercado, existem outras implementações do JPA, como o *EclipseLink* e o *OpenJPA*. [1, 2]

Resumo da diferença

- **JPA** = É a **regra / interface** (o "o que deve ser feito").
- **Hibernate** = É o **código / motor** que executa isso na prática (o "como é feito"). [1, 2, 3]

Nota comum: Quando você usa o *Spring Boot* com a dependência do **Spring Data JPA**, por baixo dos panos ele geralmente usa o **Hibernate** como o motor principal para conversar com o banco de dados. [1, 2]